Banking Management Database Assignment Report

Name: Venkata Krishna Seshu Parvathaneni

ID : 25042114

1. Data Generation Methodology

To build a realistic relational database for a retail banking system without violating privacy laws or relying on third-party dataset downloads, all records were synthetically generated using a deterministic Python script incorporating the Faker library and custom validation logic.

Customer Profiles: Synthetic customer demographics (names, national IDs, email addresses, phone numbers, and physical addresses) were generated using localized generator distributions. Synthetic tax IDs and phone numbers adhere strictly to standard format masks (e.g., standard national ID lengths and telephone prefixes).

Account Numbers & IBANs: Account identifiers (10-digit account numbers and 22-character ISO 13616 compliant IBAN structures) were generated using check-digit algorithms (Modulus 97 and Luhn algorithm)to replicate production-grade banking validation.

Financial Transactions: Transaction histories were programmatically constructed to simulate real-world transactional behavior across a 12-month period:

Distribution: Deposits (30%), Withdrawals (20%), Transfers (35%), and Merchant Payments (15%).

Amounts: Formatted using standard log-normal distributions to reflect realistic retail spending (median $45.00, maximum $5,000.00 for standard retail accounts).

Timestamps: Randomly distributed with weighted probabilities peaking during standard business hours and weekend retail hours.

Synthetic Referential Integrity: Transaction references maintain strict referential integrity by selecting existing primary keys (account_id, branch_id, and employee_id) sequentially generated in memory prior to dataset export.

2. Database Schema (Textual Representation):

The relational schema for the Retail Bank Management System consists of six core entities designed in 3rd Normal Form (3NF). Primary Keys (PK) and Foreign Keys (FK) are defined below:

BRANCHES (branch_id INTEGER PRIMARY KEY AUTOINCREMENT, branch_name VARCHAR(100) NOT NULL, routing_number VARCHAR(9) UNIQUE NOT NULL, street address VARCHAR(255) NOT NULL, city VARCHAR(100) NOT NULL, postal_code VARCHAR(20) NOT NULL, phone_number VARCHAR(20) NOT NULL);

CUSTOMERS (customer_id INTEGER PRIMARY KEY AUTOINCREMENT, first_name VARCHAR(50) NOT NULL, last_name VARCHAR(50) NOT NULL, national_id VARCHAR(20) UNIQUE NOT NULL, email VARCHAR(100) UNIQUE NOT NULL, phone_number VARCHAR(20) NOT NULL, date_of_birth DATE NOTNULL, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);

ACCOUNTS (account_id INTEGER PRIMARY KEY AUTOINCREMENT, customer_id INTEGER NOT NULL, branch_id INTEGER NOT NULL, account_number VARCHAR(12) UNIQUE NOT NULL, account_type VARCHAR(20) CHECK (account_type IN

('Checking', 'Savings', 'Money Market', 'Fixed Deposit')), balance DECIMAL(15, 2) NOT NULL

DEFAULT 0.00, status VARCHAR(15) CHECK (status IN ('Active', 'Dormant', 'Frozen',

'Closed')), opened_date DATE NOT NULL, FOREIGN KEY (customer_id) REFERENCES

CUSTOMERS(customer_id) ON DELETE RESTRICT, FOREIGN KEY (branch_id) REFERENCES

BRANCHES(branch_id) ON DELETE RESTRICT);

TRANSACTIONS (transaction_id BIGINT PRIMARY KEY AUTOINCREMENT, account_id INTEGER NOT NULL, transaction_type VARCHAR(20) CHECK (transaction_type IN ('Deposit', 'Withdrawal', 'Transfer_In', 'Transfer_Out', 'Fee')), amount DECIMAL(15, 2) CHECK (amount > 0), timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP, description VARCHAR(255), reference_code VARCHAR(36) UNIQUE NOT NULL, FOREIGN KEY (account_id) REFERENCES ACCOUNTS(account_id) ON DELETE RESTRICT); LOANS (loan_id INTEGER PRIMARY KEY AUTOINCREMENT, customer_id INTEGER NOT NULL, principal_amount DECIMAL(15, 2) CHECK (principal_amount > 0), interest_rate DECIMAL(5, 2)CHECK (interest_rate >= 0), term_months INTEGER CHECK (term_months > 0), start_date DATE NOT NULL, status VARCHAR(20) CHECK (status IN ('Pending', 'Active', 'Defaulted', 'Paid_In_Full')), FOREIGN KEY (customer_id) REFERENCES CUSTOMERS(customer_id) ON DELETE RESTRICT); CARDS (card_id INTEGER PRIMARY KEY AUTOINCREMENT, account_id INTEGER NOT NULL, card_number_hash VARCHAR(64) UNIQUE NOT NULL, card_type VARCHAR(10) CHECK (card_type IN

('Debit', 'Credit')), expiration_date DATE NOT NULL, status VARCHAR(15) CHECK (status IN

('Active', 'Blocked', 'Expired')), FOREIGN KEY (account_id) REFERENCES ACCOUNTS(account_id)ON DELETE CASCADE);

3. Normalisation & Table Justification

The architecture decomposes core banking services into normalized entities up to Third Normal Form (3NF) to eliminate data redundancy and prevent insertion, update, and deletion anomalies:

Separation of CUSTOMERS and ACCOUNTS (1:N): A single customer may maintain multiple accounts (e.g.,Checking and Savings). Combining profile data with account records would duplicate customer contact details for every new account created (violating 2NF).

Separation of ACCOUNTS and TRANSACTIONS (1:N): Storing transaction log arrays directly within an account record violates 1NF (atomicity). Splitting transactions into a dedicated ledger table supports append-only transaction tracking and sub-second querying for balance audits.

Separation of BRANCHES: Branch contact numbers and routing codes depend solely on the branch itself, not on individual account holders. Normalizing branches avoids transitive dependencies (violating 3NF).

Dedicated LOANS & CARDS Entities: Credit facilities and physical cards operate under independent lifecycles, approval workflows, and constraints separate from deposit accounts.

4. Attribute Constraints & Data Integrity:

Table	Attribute	Constraint Type	Business / Technical Justification
CUSTOMERS	national_id	UNIQUE, NOT NULL	Prevents duplicate customer identity creation; enforces Know Your Customer (KYC) identity compliance.
ACCOUNTS	balance	DECIMAL(15,2)	Avoids floating-point rounding errors native to FLOAT or DOUBLE during financial ledger balance calculations.
ACCOUNTS	status	CHECK ENUM	Restricts account states strictly to system-valid statuses ('Active', 'Dormant', 'Frozen', 'Closed').
TRANSACTIONS	amount	CHECK (> 0)	Enforces strictly positive transaction values. Debit/credit direction is explicitly managed by transaction_type.
CARDS	card_number_hash	UNIQUE, NOT NULL	Ensures raw Primary Account Numbers (PANs) are never stored unencrypted in database tables (PCI-DSS compliance).